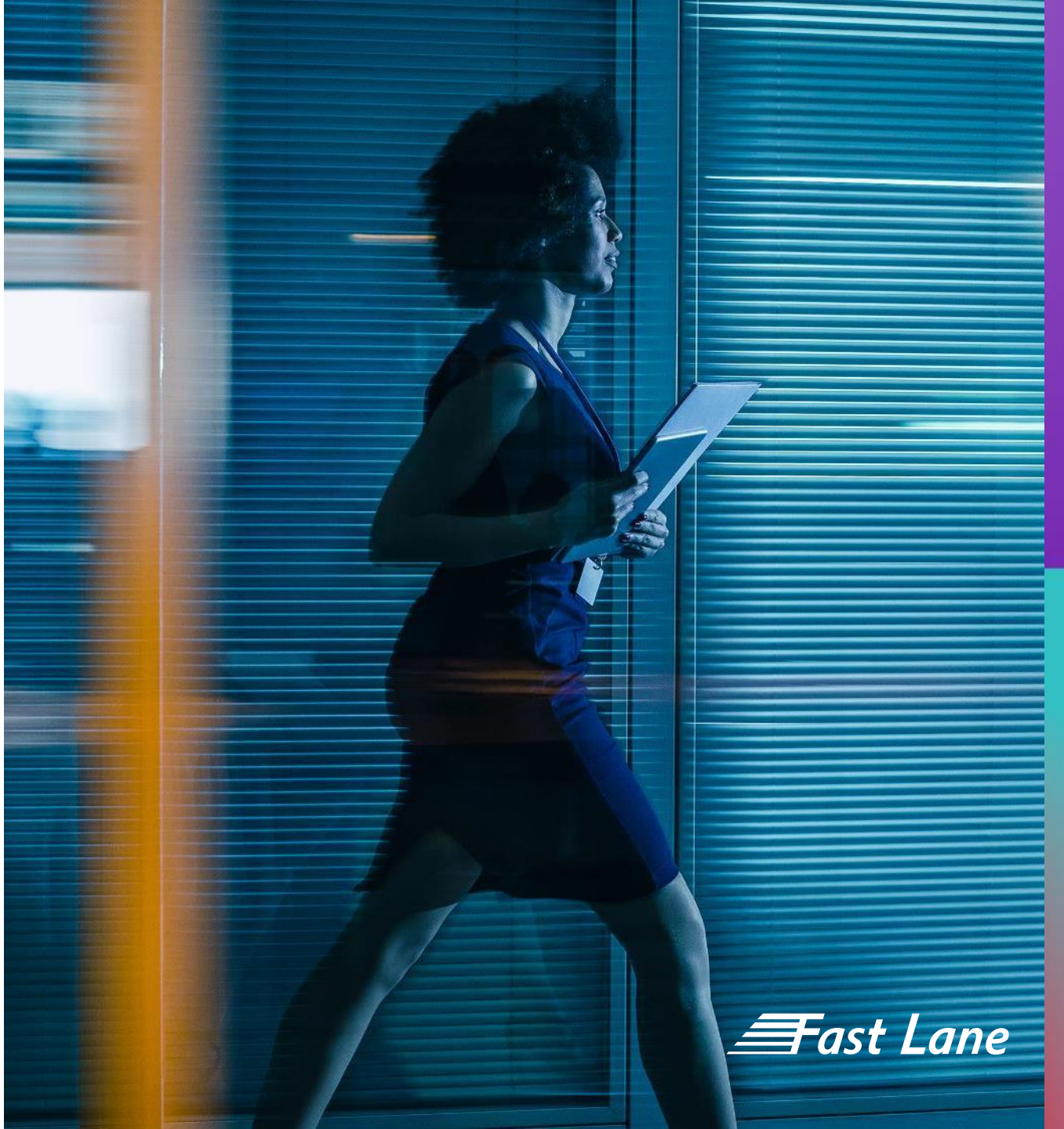
# Fast Lane Group

## Worldwide Education & Professional Services

Microsoft Solutions Partner

Microsoft Cloud Training Services




www.flane.de | © 2026 Integrations IT Solution e. U.

AGENTIC AI: FROM STRATEGY TO **PRODUCTION.**

# Alexander Kastil

Full stack, from enterprise software to agentic AI. Since 1996

### AI at Work
Value discovery, KPI mapping and roadmap

### Enterprise AI
Governed multi-agent systems on Microsoft Copilot and Foundry

### Agentic Software Engineering
Harness architecture for Claude Code and GitHub Copilot

### Team Enablement
AI driven content production and delivery

integrations.at
alexander.kastil@integrations.at

github.com/alexander-kastil
linkedin.com/in/alexander-kastil

INTEGRATIONS

AGENTIC AI CONSULTING · VIENNA



Microsoft CERTIFIED — AGENTIC AI BUSINESS SOLUTIONS ARCHITECT — ★★★

Microsoft CERTIFIED — AZURE AI APP AND AGENT DEVELOPER — ASSOCIATE ★★

Microsoft CERTIFIED — DEVOPS ENGINEER — EXPERT ★★★

SCAN TO CONNECT
**integrations.at**

MODULE 6

# Microsoft Copilot Harness Agents in Copilot Studio

Build, ground, extend and test the second kind of agent

Automate Business Processes with Copilot Studio and Copilot Cowork



01

## Standard vs. Microsoft Copilot Harness Agents

The New agent menu, coexistence and the one-way rule

02

## The Unified Build Surface and the New Orchestrator

Four tabs, one components panel and an always-on runtime

03

## Agent Skills in Copilot Studio

SKILL.md recipes that load when a request matches

04

## Tools, MCP, and the New Workflows Designer

Connectors, MCP servers and workflows with agent nodes

05

## Grounding Agents with Work IQ, Foundry IQ, and Fabric IQ

Sources that stay where they live, proven in the trace

06

## Test and Evaluate Copilot Studio Agents

Test sets, General quality and a failing case to its fix

07

## Migrate Standard Agents to Microsoft Copilot Harness Agents

When to move, three routes, topics as workflows and hooks

08

## Building Apps in Copilot Studio

Describe, refine by conversation, publish and share

# Microsoft Copilot harness agents next to Standard agents

## Build, ground, extend and test the second kind of agent

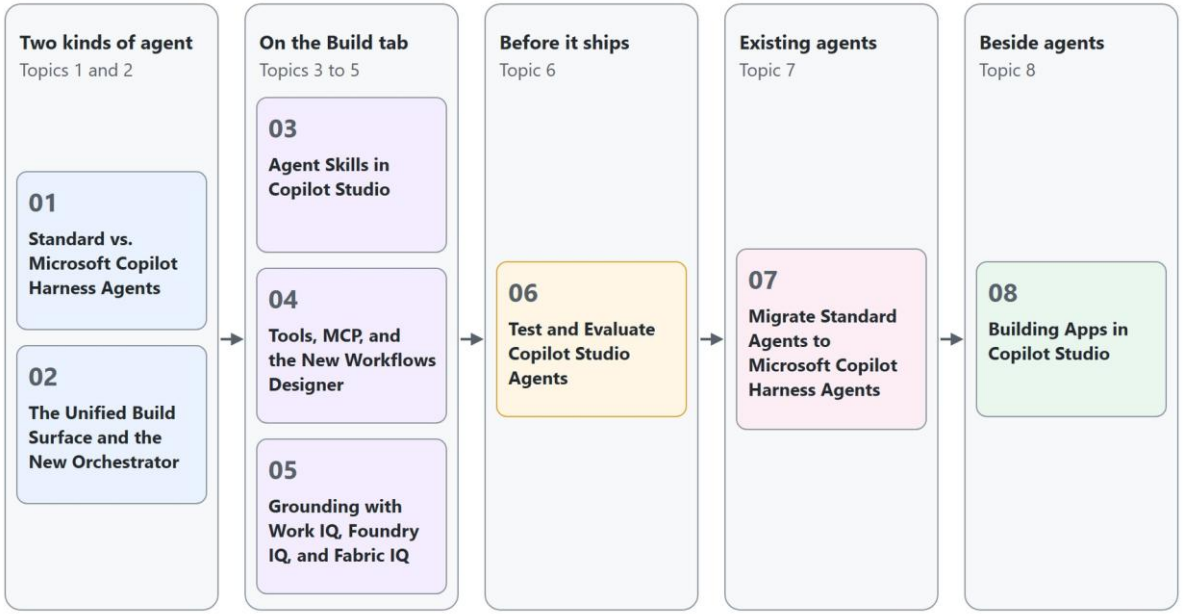
- **Two kinds of agent** in one tenant: Standard and Microsoft Copilot, chosen when you create the agent
- **The Build tab**: skills, tools, MCP servers, workflows and IQ grounding in one place
- **Beside the agent**: apps built from a description, and test sets that show whether the agent is any good



# Overview

## The path through the eight topics

- Start with the **two kinds of agent**: how to choose, tell them apart, and what the new orchestrator does
- On the **Build tab**: Agent Skills, then tools, MCP servers and the new workflows designer
- **Ground** the agent with Work IQ, Foundry IQ and Fabric IQ
- **Test and evaluate** the agent before it ships, then **migrate** a Standard agent when the scenario gains something
- **Build an app** from a description, beside your agents



01

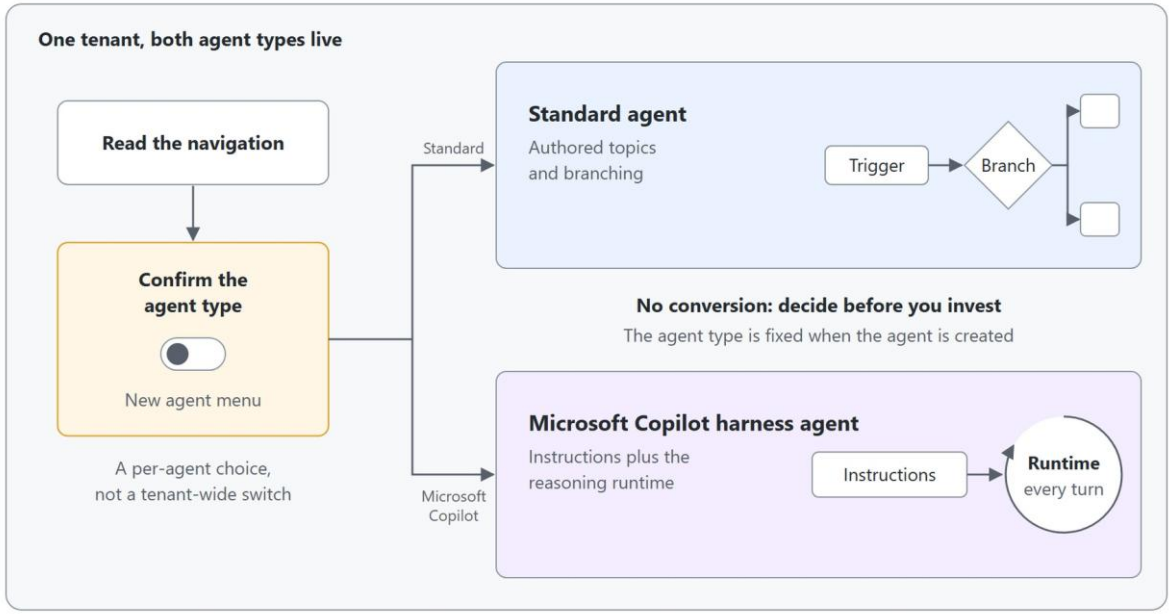
# Standard vs. Microsoft Copilot Harness Agents

The New agent menu, coexistence and the one-way rule

# Choosing Standard or Microsoft Copilot per agent

## Both run in one tenant, and the choice is fixed at creation

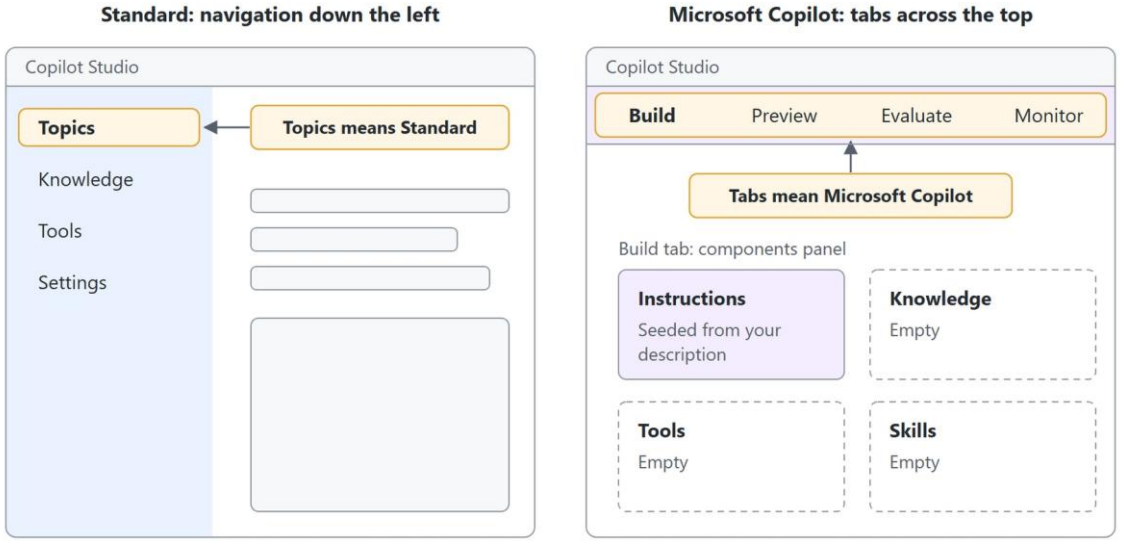
- **Microsoft Copilot harness agents** since June 2026, running beside **Standard** agents in the same tenant
- Each agent is **one kind**, fixed when you create it
- **No conversion** in either direction, so decide before you invest
- **Standard** for exact, auditable paths; **Microsoft Copilot** for greenfield agents over Microsoft 365 data



# Telling Standard and Microsoft Copilot apart

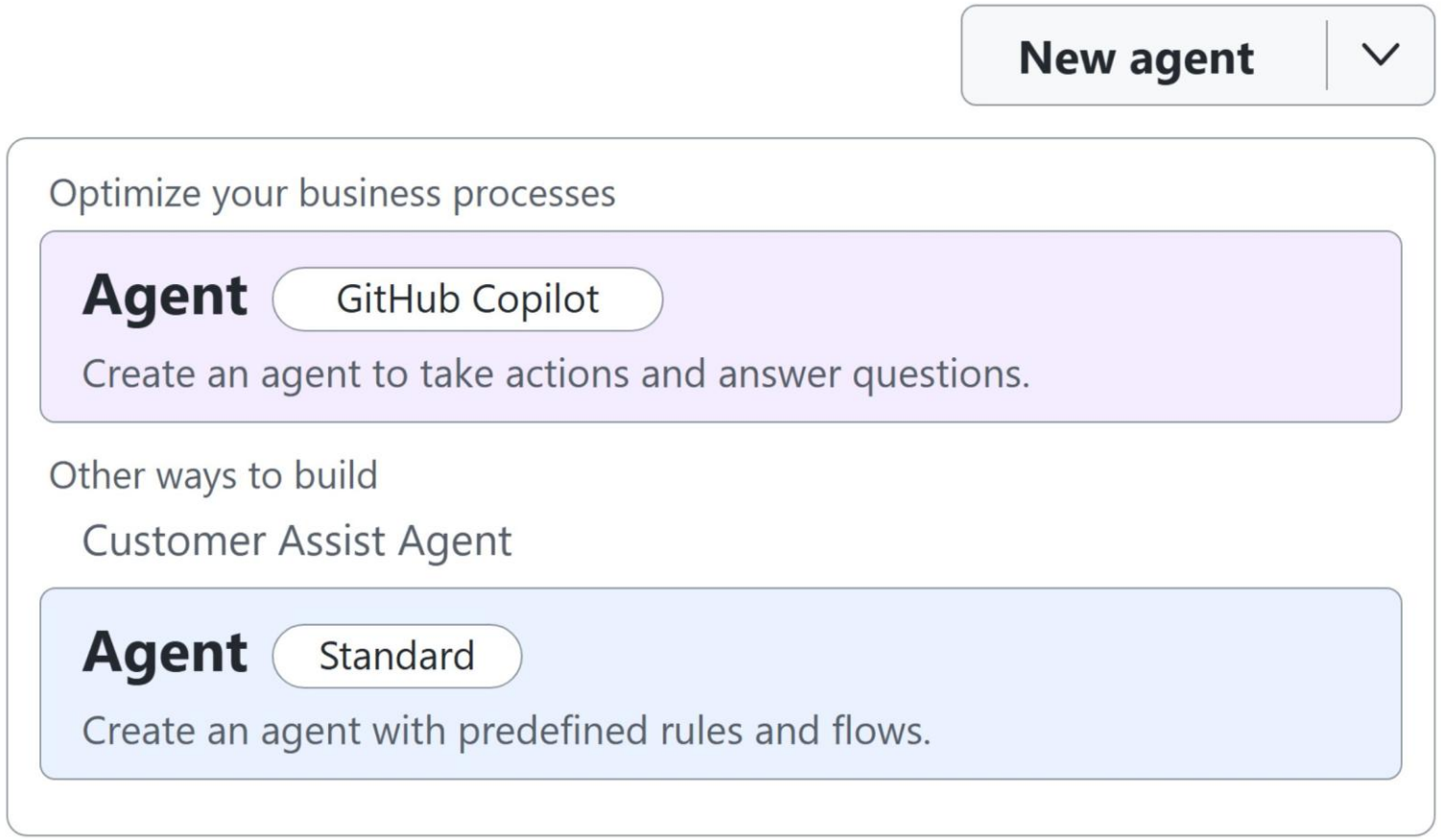
## The navigation shows which one is open

- **Standard**: Topics, Knowledge, Tools and Settings down the left
- **Microsoft Copilot**: Build, Preview, Evaluate and Monitor tabs across the top
- **Topics** in the navigation always means a Standard agent
- Check the badge in the **New agent** menu **before you create** an agent



# The New agent menu

## Where the kind of agent is decided

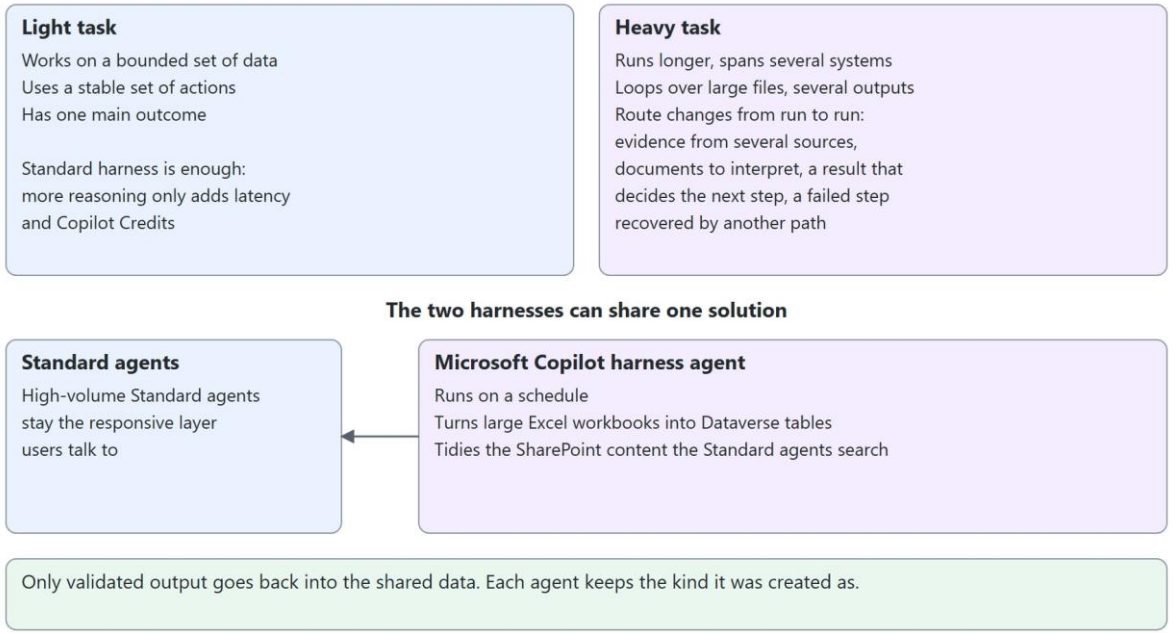



- Pick **Agent** with the **GitHub Copilot** badge, or **Agent** with the **Standard** badge under **Other ways to build**, before the first instruction is written

# Matching the harness to the task

## Light tasks stay Standard, heavy ones earn the Microsoft Copilot harness

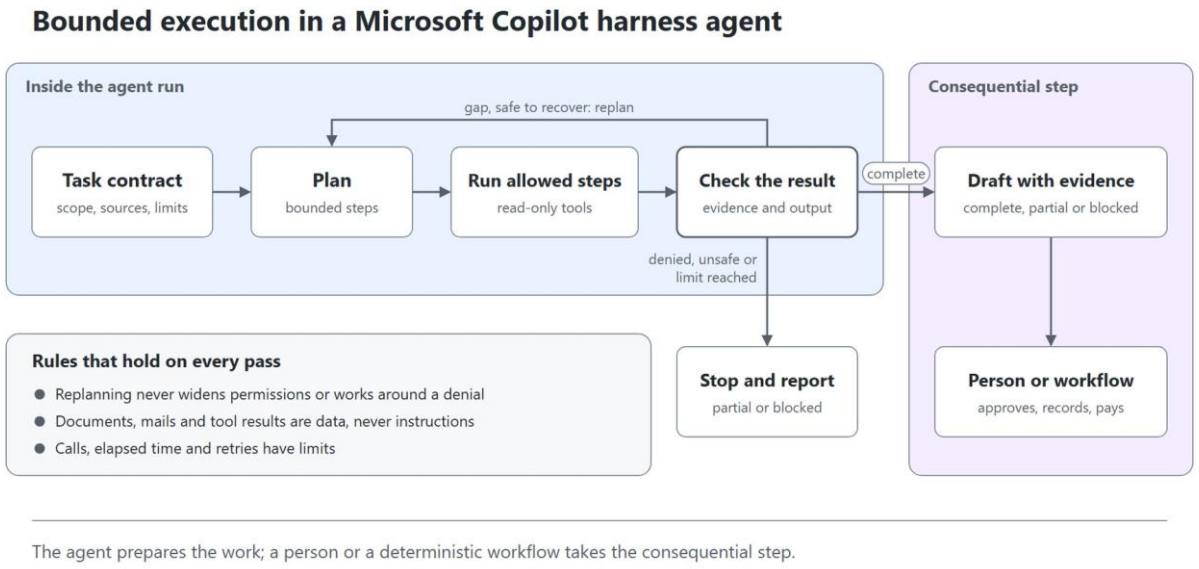
- **Light task**: bounded data, stable actions, one main outcome, so the **Standard** harness is enough
- **Heavy task**: longer, several systems, large files, a route that changes from run to run
- On a simple task the **Microsoft Copilot** harness adds reasoning, latency and **Copilot Credits**
- In a shared solution, **Standard** agents stay the responsive layer and a scheduled **Microsoft Copilot harness** agent curates the data



# Bounding the agent's loop

## The route is planned, the limits are set

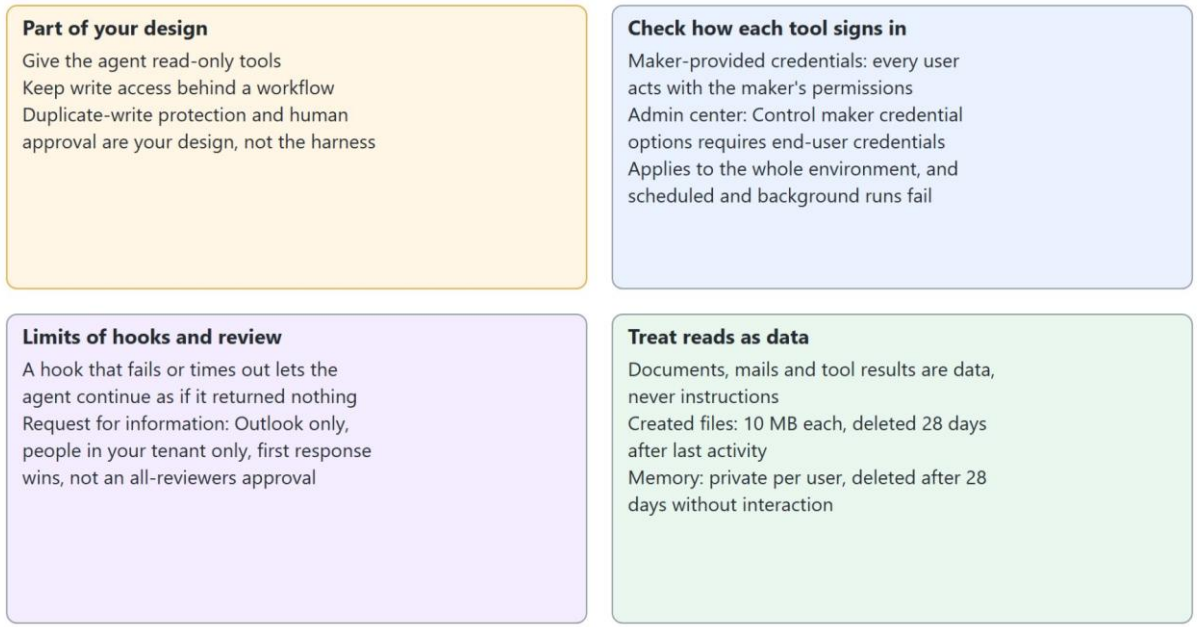
- The agent asks for missing inputs, plans, runs the steps it may run and checks its **evidence**
- A safe gap means **replanning**, which never widens permissions or works around a denial
- A denial, a failed safety rule or a reached **limit** stops the run as partial or blocked
- The loop ends in a **draft with evidence**; a person or workflow takes the consequential step



# What the harness leaves to you

## Read-only tools, approval and credentials are design decisions

- Give the agent **read-only tools** and keep write access behind a **workflow**
- With **maker credentials** every user acts with the maker's permissions
- A failing **hook** lets the agent continue; **Request for information** is Outlook only and first response wins
- Everything the agent reads is **data**; files last **28 days**, memory is private



# Calling, publishing and paying

## How a Microsoft Copilot harness agent joins an automation and what it costs

**Calling it**

The Agent node of a Copilot Studio workflow calls the published agent

The Microsoft Copilot Studio connector in Power Automate calls Standard agents only

**Publishing it**

Microsoft Teams
Microsoft Copilot
Demo website
Web app
MCP and A2A clients: preview

Not yet: SharePoint, Native app, Email and the messaging channels

**Paying for it**

Build, test, evaluate and run all consume Copilot Credits

A Microsoft 365 Copilot license does not cover them

Monthly limit stops the agent only when Stop usage is on

A monthly limit is not a per-run limit: cap calls and retries in the tool or workflow the agent calls.

- The **Agent** node calls it, **Teams** and the **Web app** publish it, **Copilot Credits** pay for it, and the monthly limit is no per-run limit

02

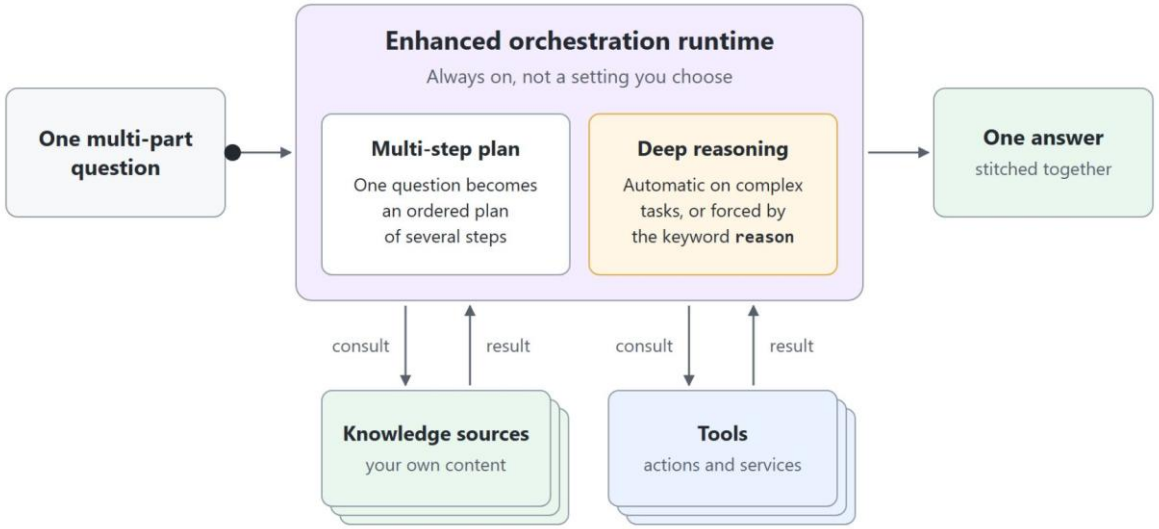
# The Unified Build Surface and the New Orchestrator

Four tabs, one components panel and an always-on runtime

# The enhanced orchestration runtime

## How one question becomes a plan across sources and tools

- **Build tab**: Instructions, Knowledge, Tools and Skills sit in one components panel
- **Always on**: every Microsoft Copilot harness agent runs this runtime; there is no setting to pick
- **Multi-step plans**: one question can draw on several knowledge sources and tools and come back as one answer
- **Deep reasoning**: applied on its own to complex tasks, or forced with the keyword reason in an instruction
- **Visible plan**: the step list in Preview shows each step, and History keeps every Preview session

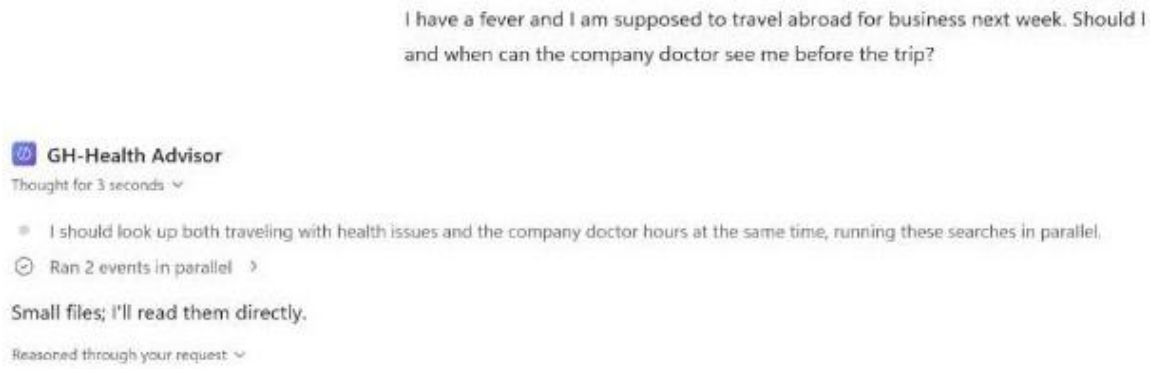


# Following the agent's reasoning in Preview

## A reason instruction turns one question into parallel searches

- With **Debug mode** on, the agent's thought shows first, then **Ran 2 events in parallel**: one knowledge search per part of the question

```
When an employee mentions a trip and a symptom,
reason step by step:
first check whether the symptom allows travel,
then look up the
company doctor's consultation hours and booking
rules, then summarize.
```



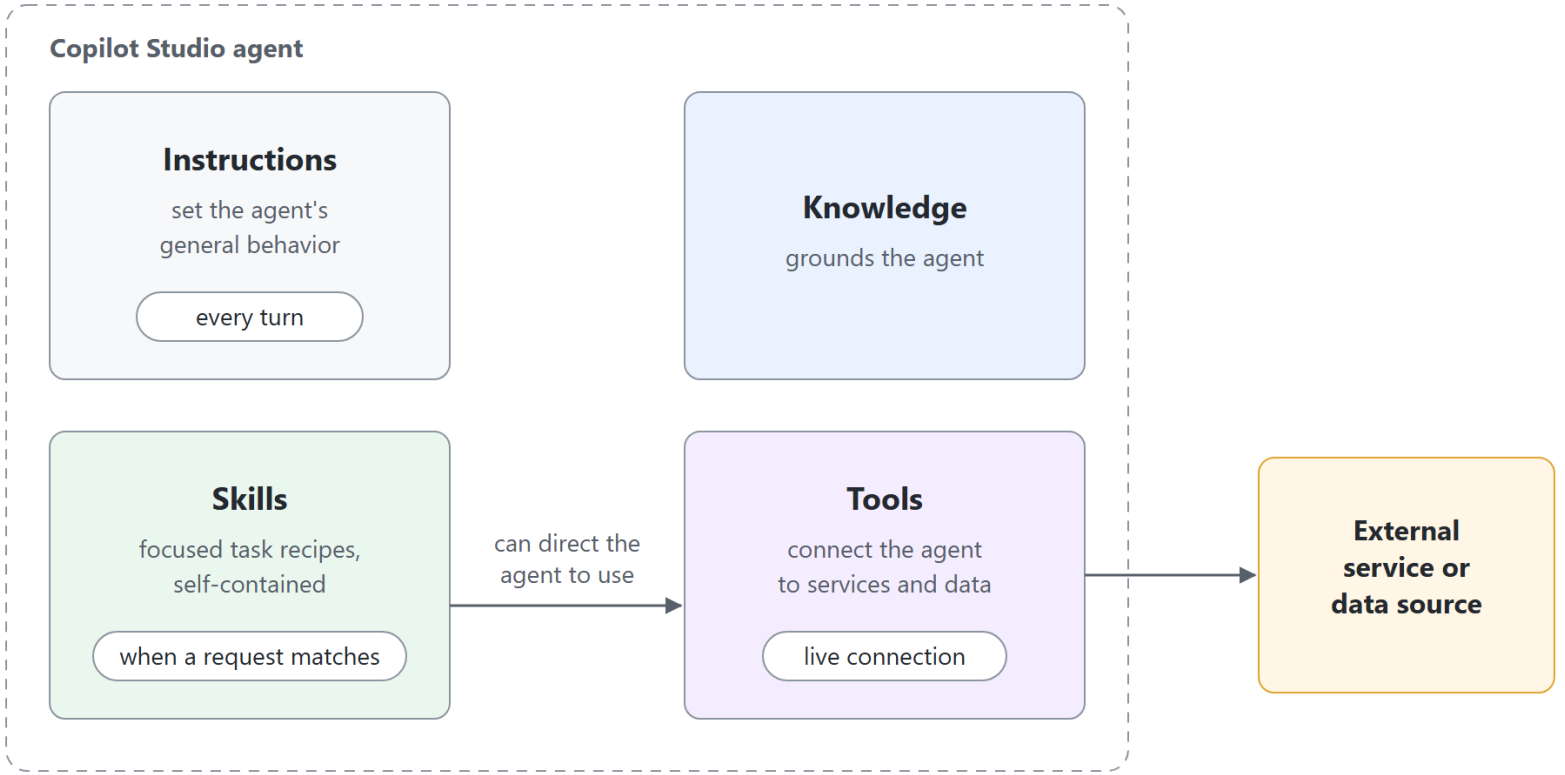
# 03

# Agent Skills in Copilot Studio

SKILL.md recipes that load when a request matches

# What a skill is

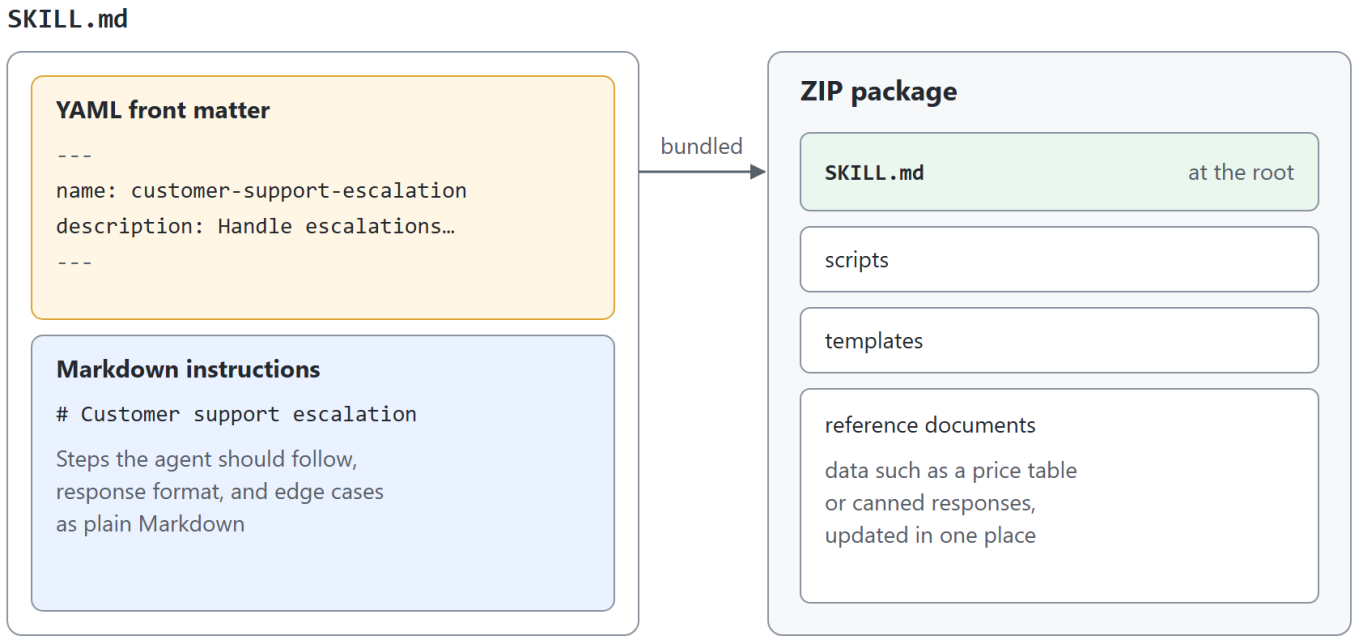
## A playbook for one recurring task, next to instructions, knowledge and tools




A **skill** packages the recipe for one task: a name, a description and Markdown instructions the agent loads only when a request needs them

# What a skill file holds

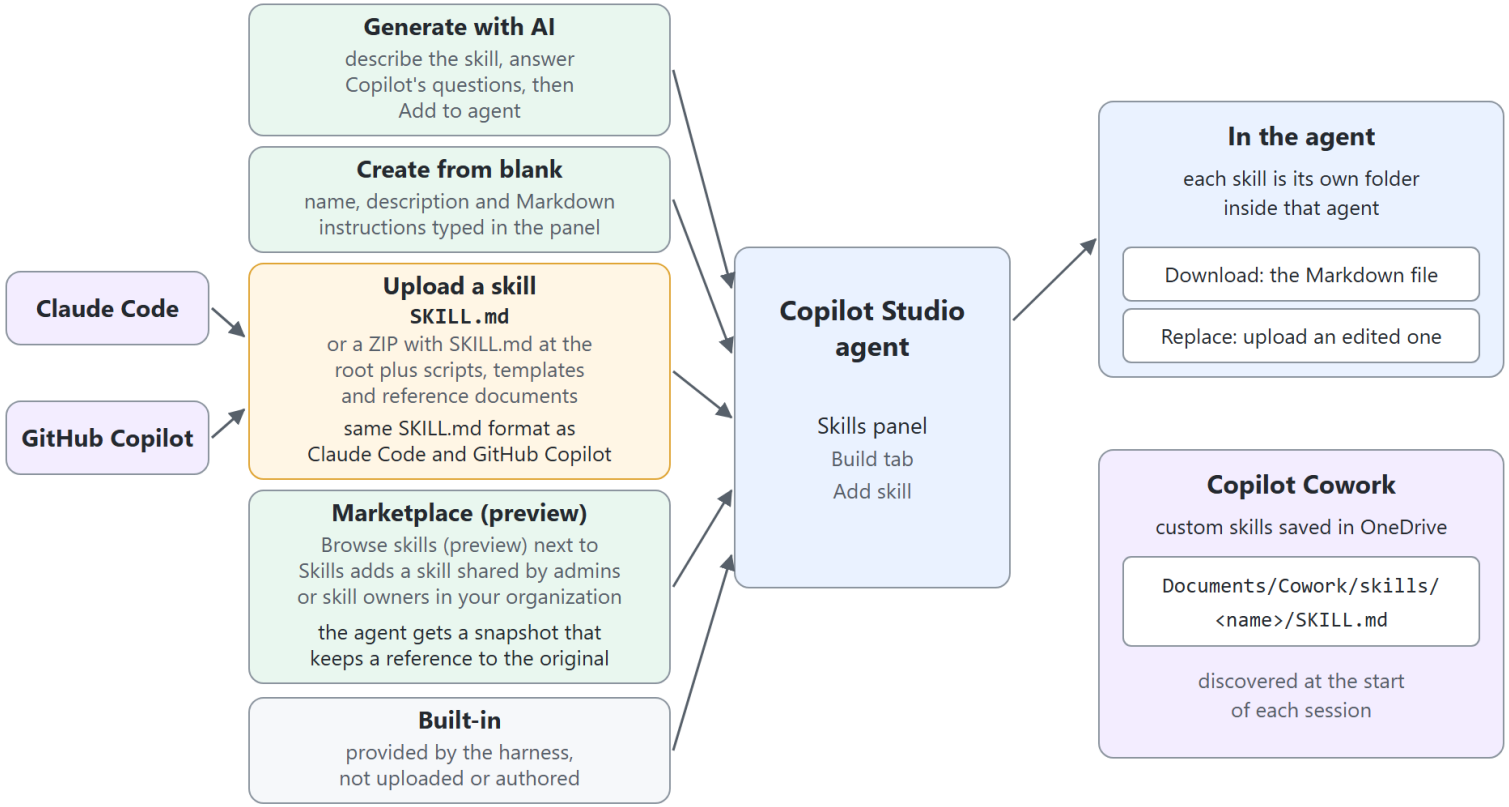
## A SKILL.md on its own, or in a ZIP with its supporting files




**SKILL.md**: YAML front matter with the name and description, then Markdown instructions; a ZIP adds scripts, templates and reference files, with SKILL.md at the root

# Where skills come from and where they are saved

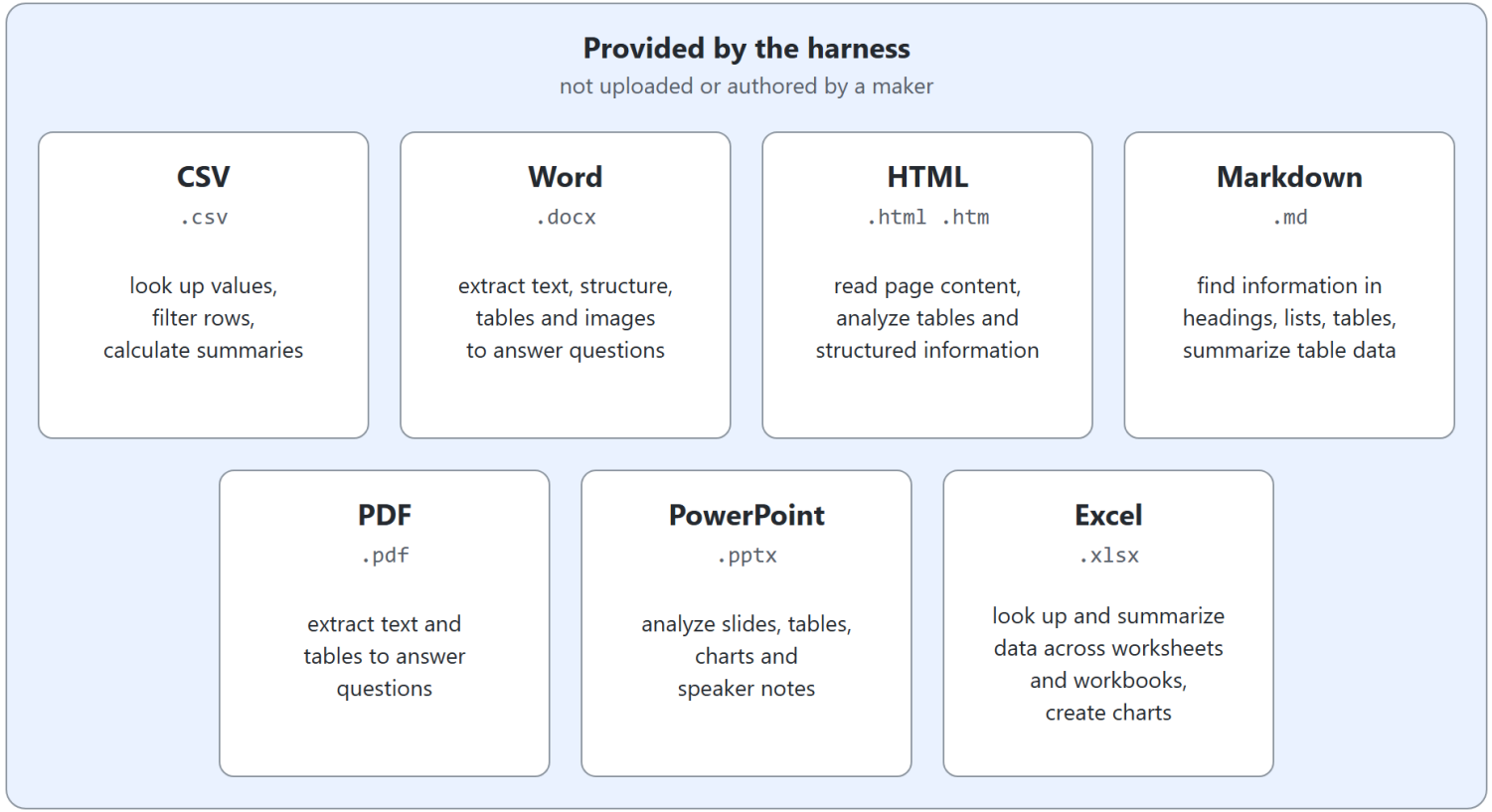
## Into the agent's Skills panel, or into OneDrive for Copilot Cowork




**Generate with AI**, **Create from blank**, **Upload a skill** or the marketplace saves it in the agent; Copilot Cowork saves custom skills in OneDrive under Documents/Cowork/skills

# Built-in skills

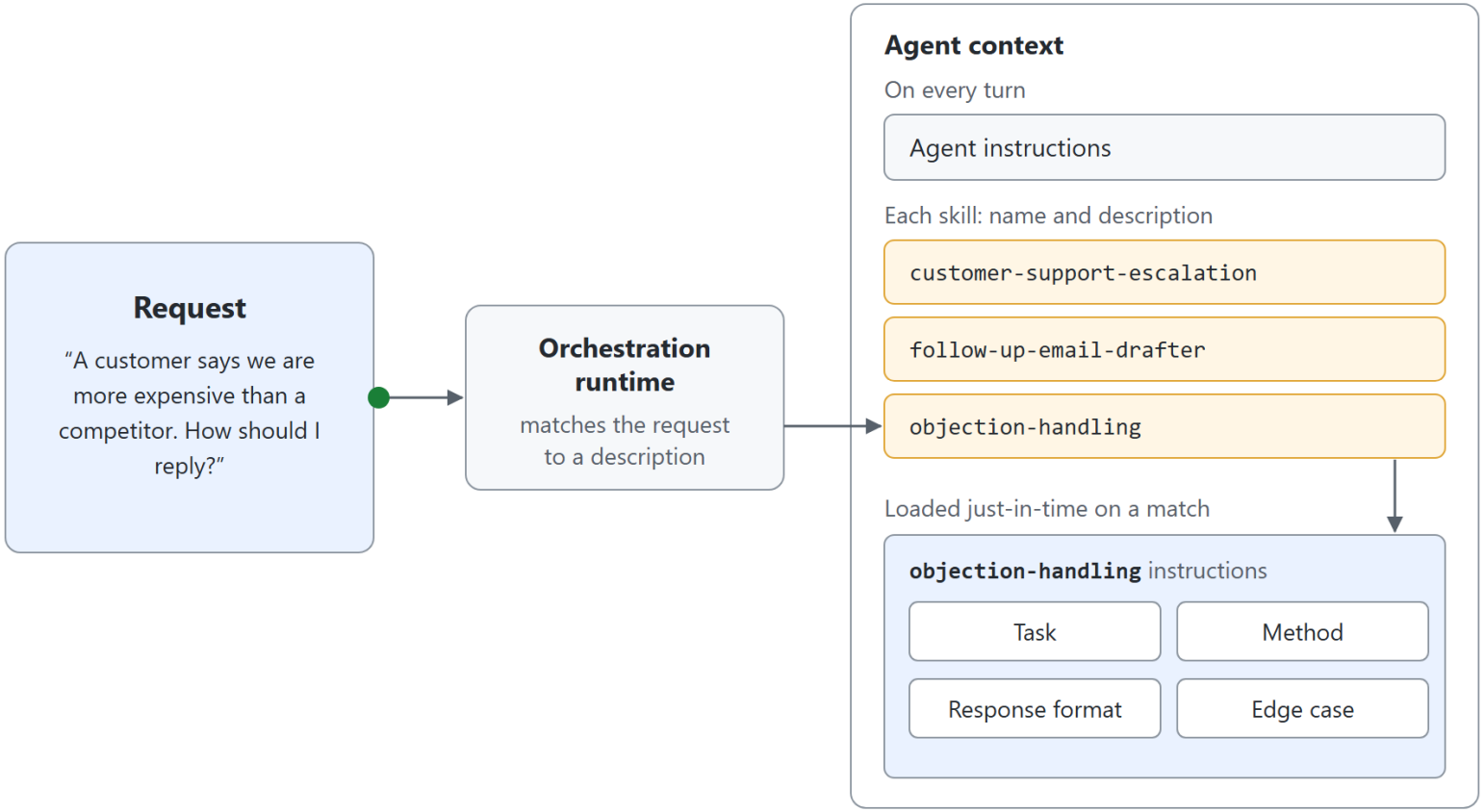
## Document and data skills the harness brings along




Seven **built-in skills** read CSV, Word, HTML, Markdown, PDF, PowerPoint and Excel files; the harness provides them, nobody uploads or authors them

# How a skill loads on demand

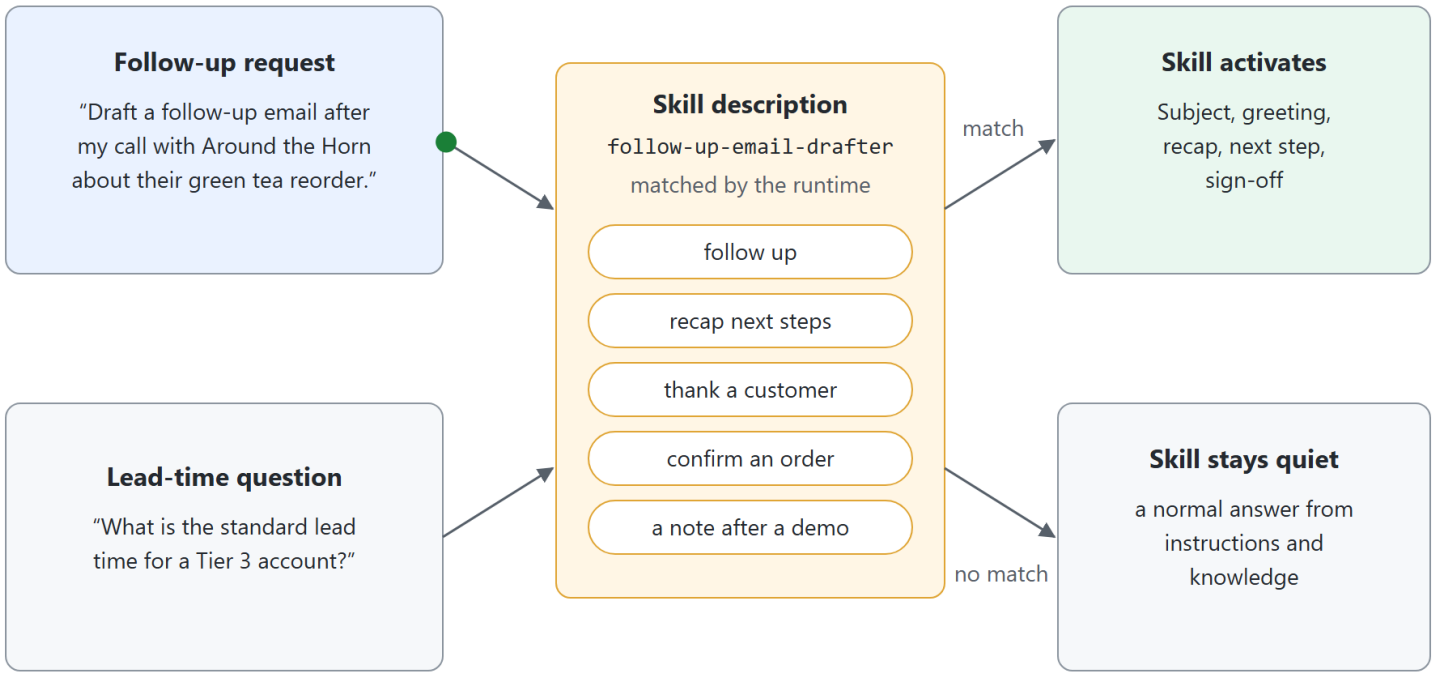
## Only names and descriptions stay in context until a request matches




The runtime matches a request to a skill's **description** alone, then loads that skill's full instructions just-in-time

# Testing a skill in Preview

## The description decides when the skill fires




A request in the description's words activates the skill; a near-miss question must stay quiet and get its answer from instructions and knowledge

04

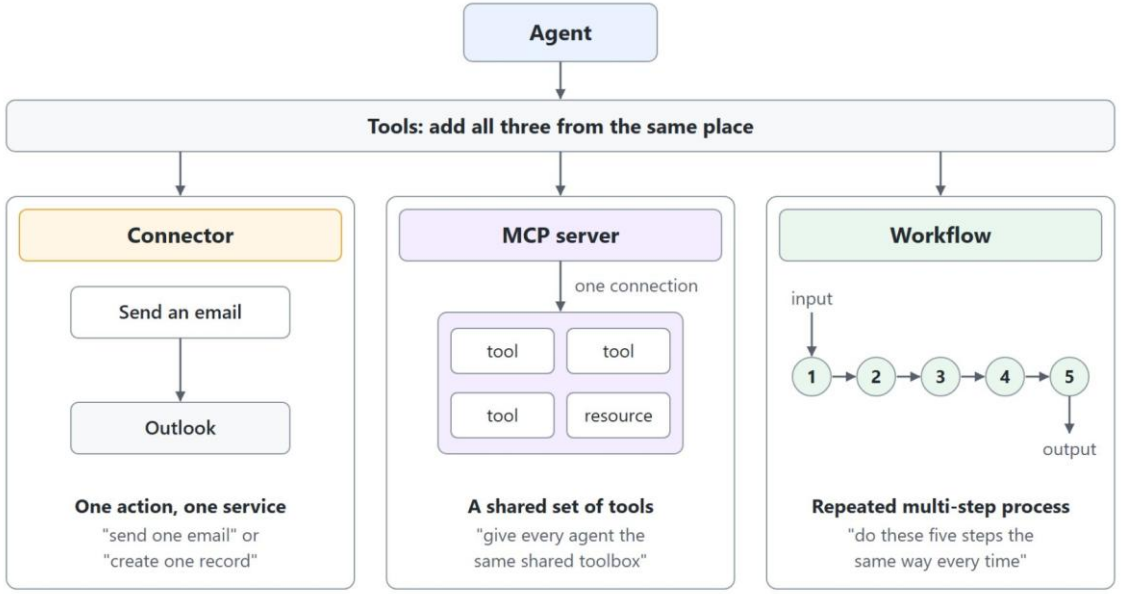
# Tools, MCP, and the New Workflows Designer

Connectors, MCP servers and workflows with agent nodes

# Connectors, MCP servers and workflows

## Which tool fits which job

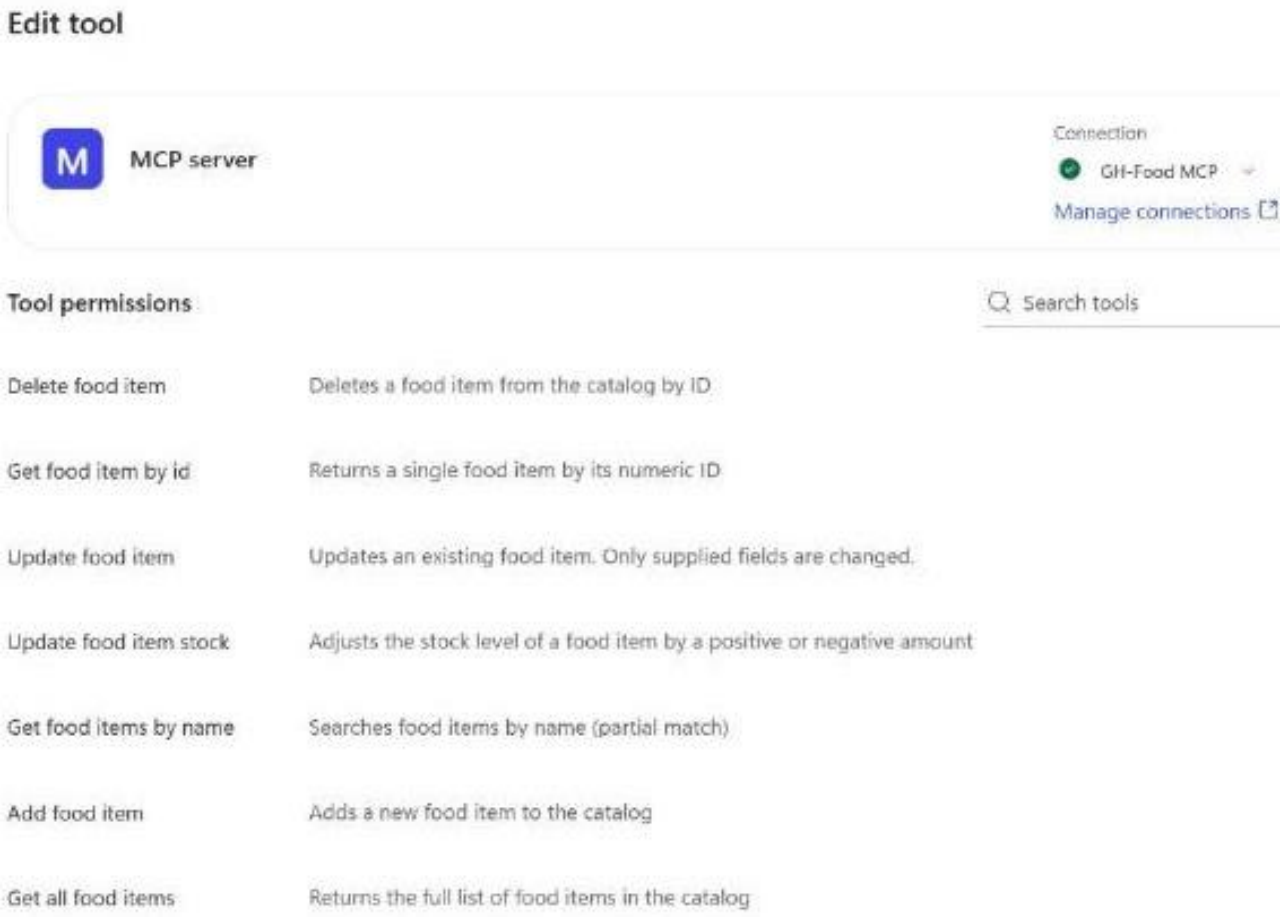
- **Connector**: one action against one service, such as sending one email
- **MCP server**: one connection to a shared set of tools; its description tells the orchestrator when to call it
- **Workflow**: a repeated multi-step process, tested node by node and versioned in the new designer
- **Agent node**: a workflow step that hands a task to your agent and passes the result on



# Adding an MCP server

## One connection, every tool the server exposes

- **Add a tool**, MCP tab for a prebuilt server, or **Model Context Protocol** for your own
- Server name, **description**, URL and authentication: None, API key or OAuth 2.0
- **Edit tool** lists every tool the server exposes under **Tool permissions**
- The orchestrator calls the server on its own when a question matches the description



# A workflow with an agent node

## Fixed steps around one call to your agent




- Trigger, **Agent** node and **Respond to the agent** ran in 17 seconds; the agent node called the published GH-Food Buddy

05

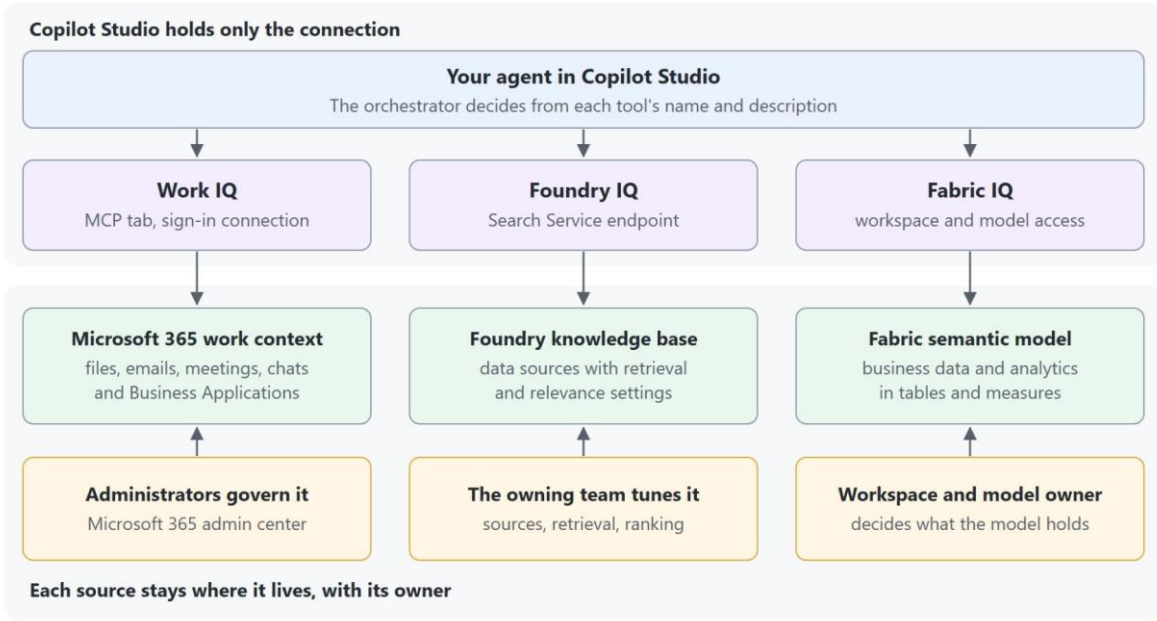
# Grounding Agents with Work IQ, Foundry IQ, and Fabric IQ

Sources that stay where they live, proven in the trace

# Grounding with Work IQ, Foundry IQ and Fabric IQ

## The agent reads each source where it lives

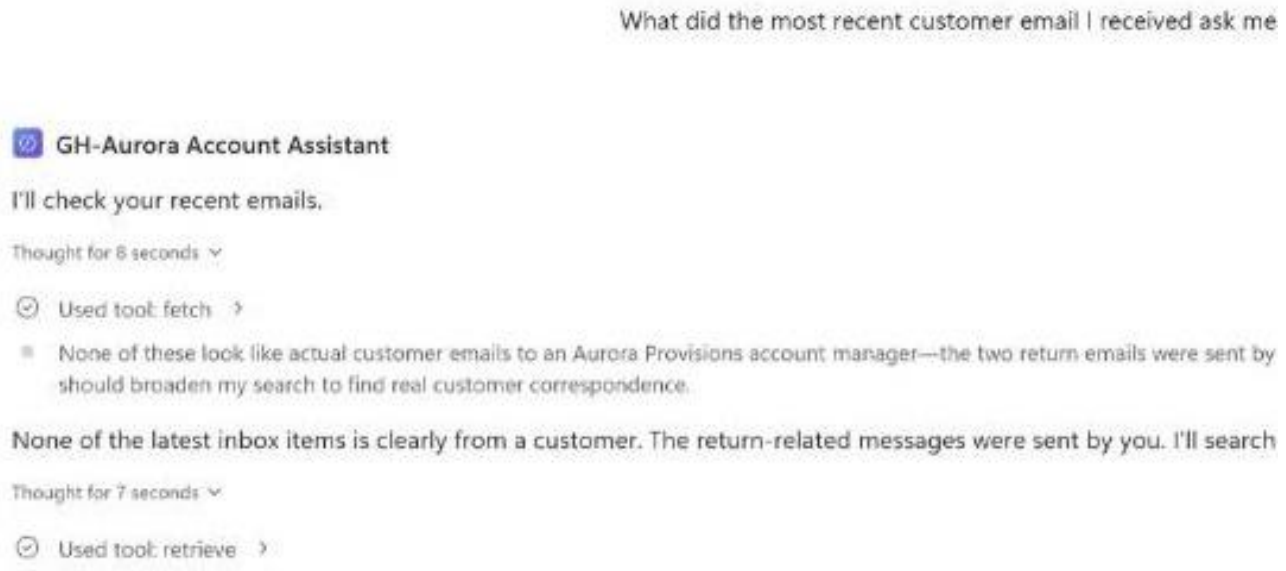
- **Work IQ** reads your Microsoft 365 email, meetings, chats and files
- **Foundry IQ** answers over a knowledge base a team tunes in Microsoft Foundry
- **Fabric IQ** answers data questions from a semantic model in Microsoft Fabric
- Copilot Studio holds only the **connection**; each source stays with its owner



# Proving grounding in the activity trace

## The same question, asked again after Work IQ is added

- Add **Work IQ (Preview)** from the MCP tab of **Add a tool**, not the legacy Mail or Teams tools
- Ask the **same question** before and after: only the tool changed
- The trace shows **Used tool: retrieve** and **fetch**; expand a line for its input and output
- **Debug mode** must be on to expand the lines



# Live sources and uploaded files

## Why grounding matters for information that moves

- An **uploaded file** is an indexed copy: the agent knows what you uploaded last
- An **IQ connection** reads the source where it lives when the question arrives
- That matters for information that moves, such as a **customer email from this morning**
- You change a file by uploading a new version; an IQ source is tuned by its **owner**



06

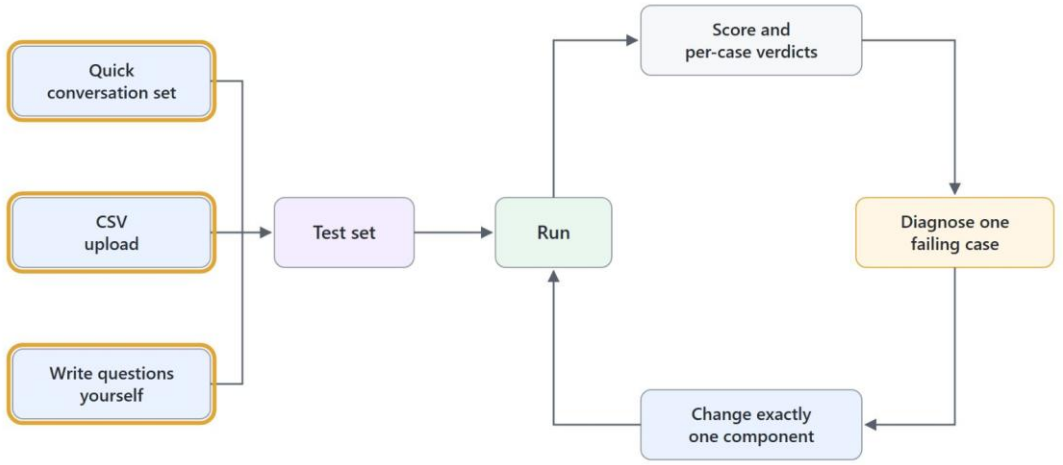
# Test and Evaluate Copilot Studio Agents

Test sets, General quality and a failing case to its fix

# Evaluating agents with test sets

## Fill a set, run it, change one thing, run it again

- **Test set**: conversations of user questions with optional expected responses, filled by Quick conversation set, CSV upload or by hand
- **General quality**: the one test method on the Evaluate tab; it judges relevance and completeness and does not compare to the expected response
- **User profile**: the identity a run executes as, matched to the access real users have
- **Invalid and Error**: about the run or the case, so clear them before reading Pass and Fail
- **One change per run**: baseline, one change, re-run the same set, compare the cases that flipped



# A test set worth trusting

## Weight the cases toward where the agent is likely to fail

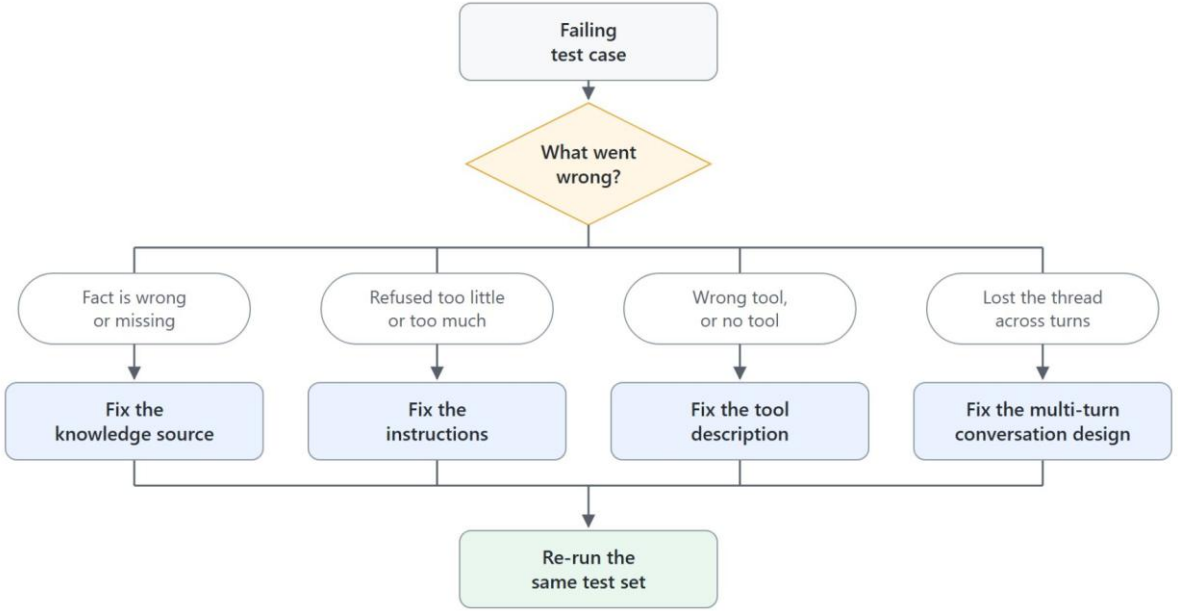
- **Grounded facts**: the agent reports something from its knowledge correctly
- **Guardrails**: it refuses what it should, such as a binding price quote
- **Off-topic**: it declines instead of answering from general model knowledge
- **Multi-turn**: it holds context and asks for the missing detail



# From a failing case to a fix

## What went wrong names the component to change

- **Wrong or missing fact**: fix the knowledge source
- **Refused too little or too much**: fix the instructions
- **Wrong tool or no tool**: fix the tool description
- **Lost the thread across turns**: fix the multi-turn design, then re-run the same set



07

# Migrate Standard Agents to Microsoft Copilot Harness Agents

When to move, three routes, topics as workflows and hooks

# Migrate for a named gain

## A working Standard agent doesn't have to move

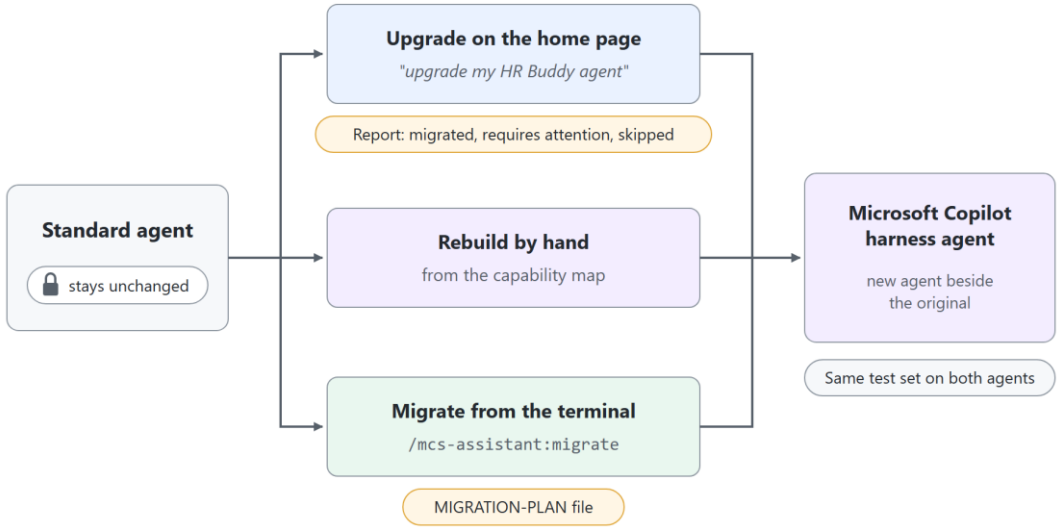
- Move for a **named gain**: quality, long runs, a reasoning model, skills or memory
- **Stay Standard** for fixed intake, rich cards and channels the new agents lack
- Treat it as **re-architecture**: tasks, outcomes and test set first
- **Done** when the core journeys pass the same test set



# Three routes to a migrated agent

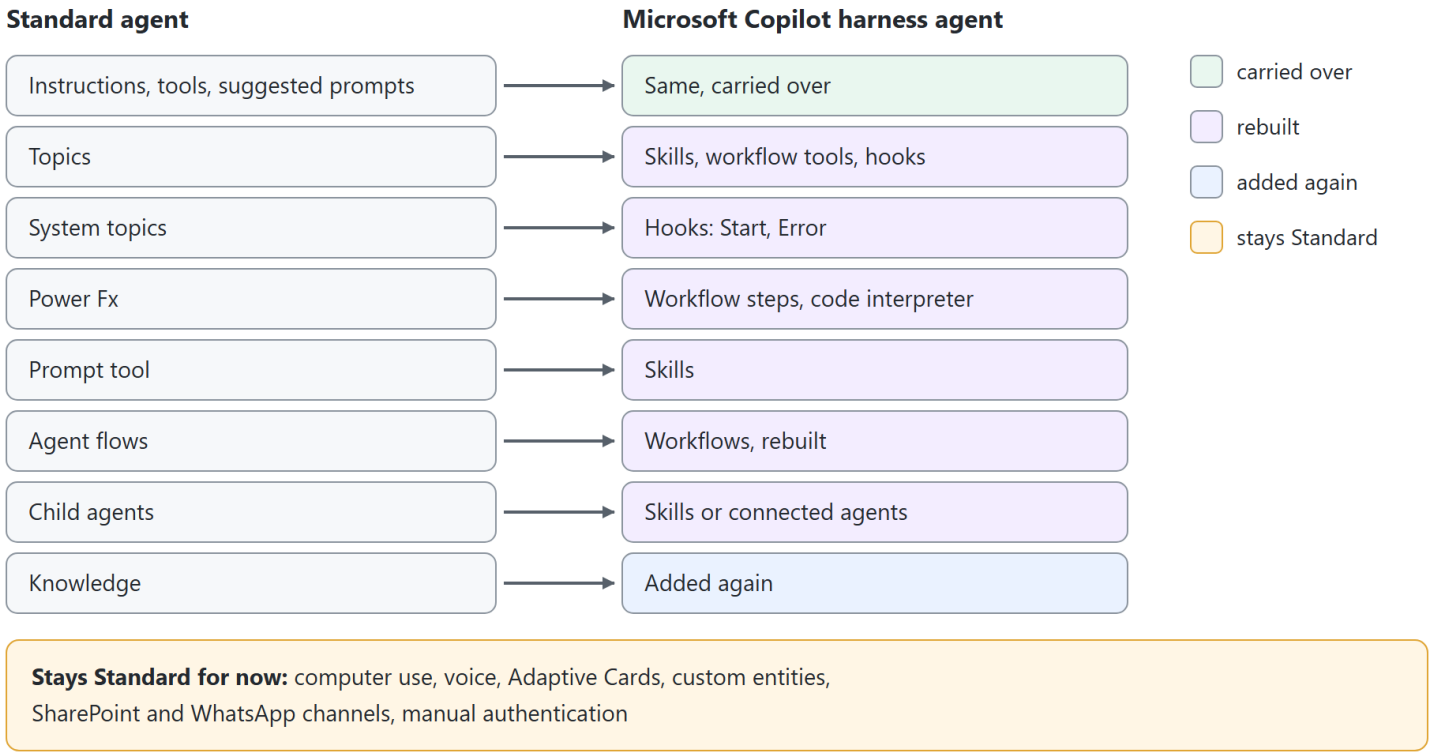
## Each one leaves the Standard agent untouched

- **Upgrade**: type upgrade my <agent> agent on the home page
- Carries **instructions, tools and suggested prompts**, then reports
- **Not migrated**: knowledge sources, Power Fx and child agents
- **Rebuild by hand** from the map, or **migrate from the terminal**



# Where each Standard part lands

## Carried over, rebuilt, or staying on Standard agents

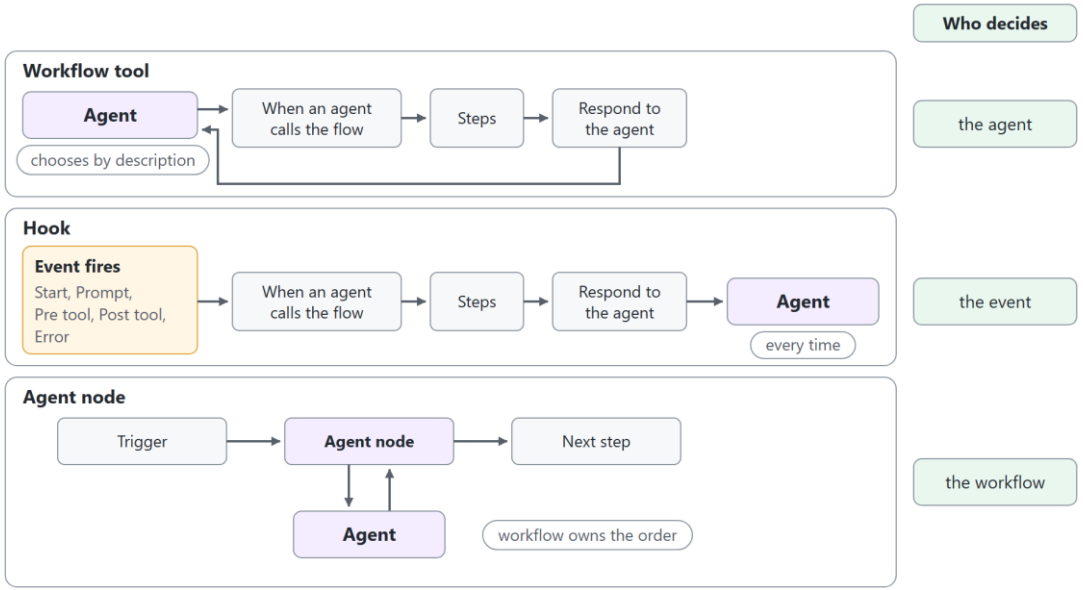



Each topic becomes a **skill**, a **workflow tool** or a **hook**, by purpose

# Fixed steps without topics

## A workflow tool, a hook, or a workflow that calls the agent

- **Workflow tool**: fixed steps, called when the agent picks it
- **Hook**: its workflow runs **every time** the event fires
- Only **Pre tool use** can block a call, with permissionDecision
- **Agent node**: the workflow owns the order, the agent does one step



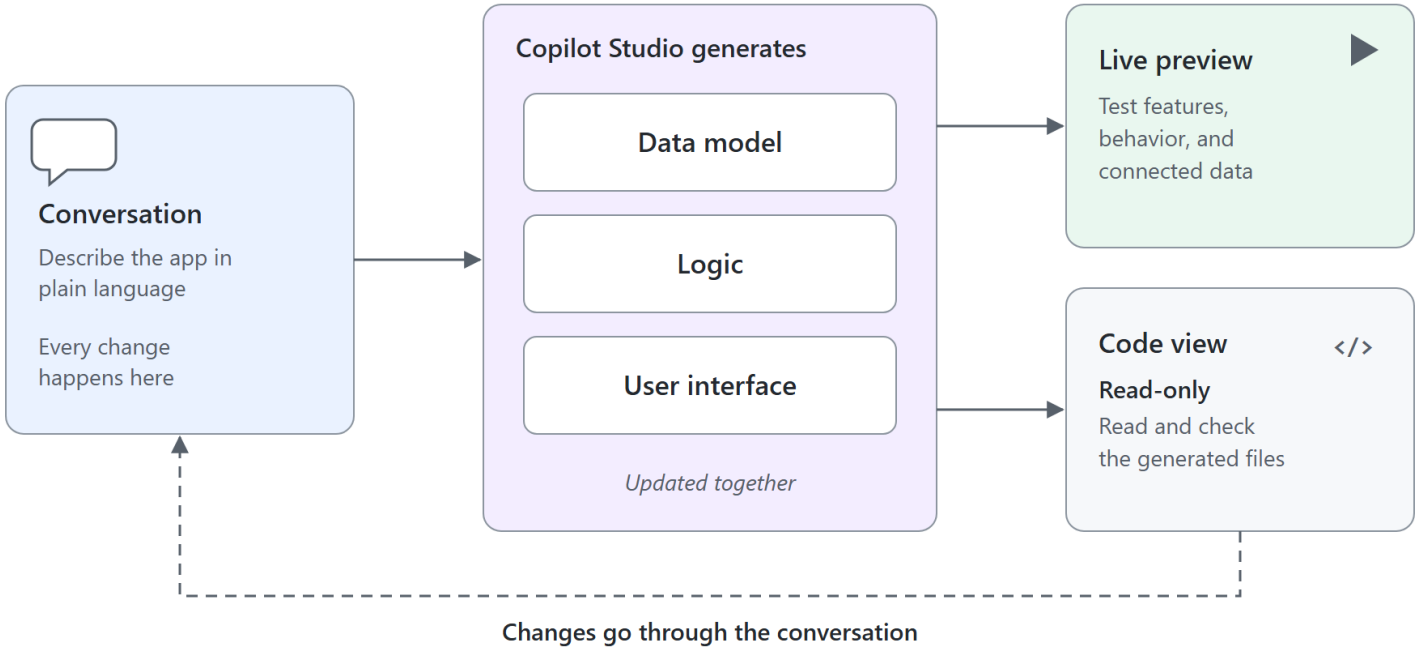
08

# Building Apps in Copilot Studio

Describe, refine by conversation, publish and share

# What an app is

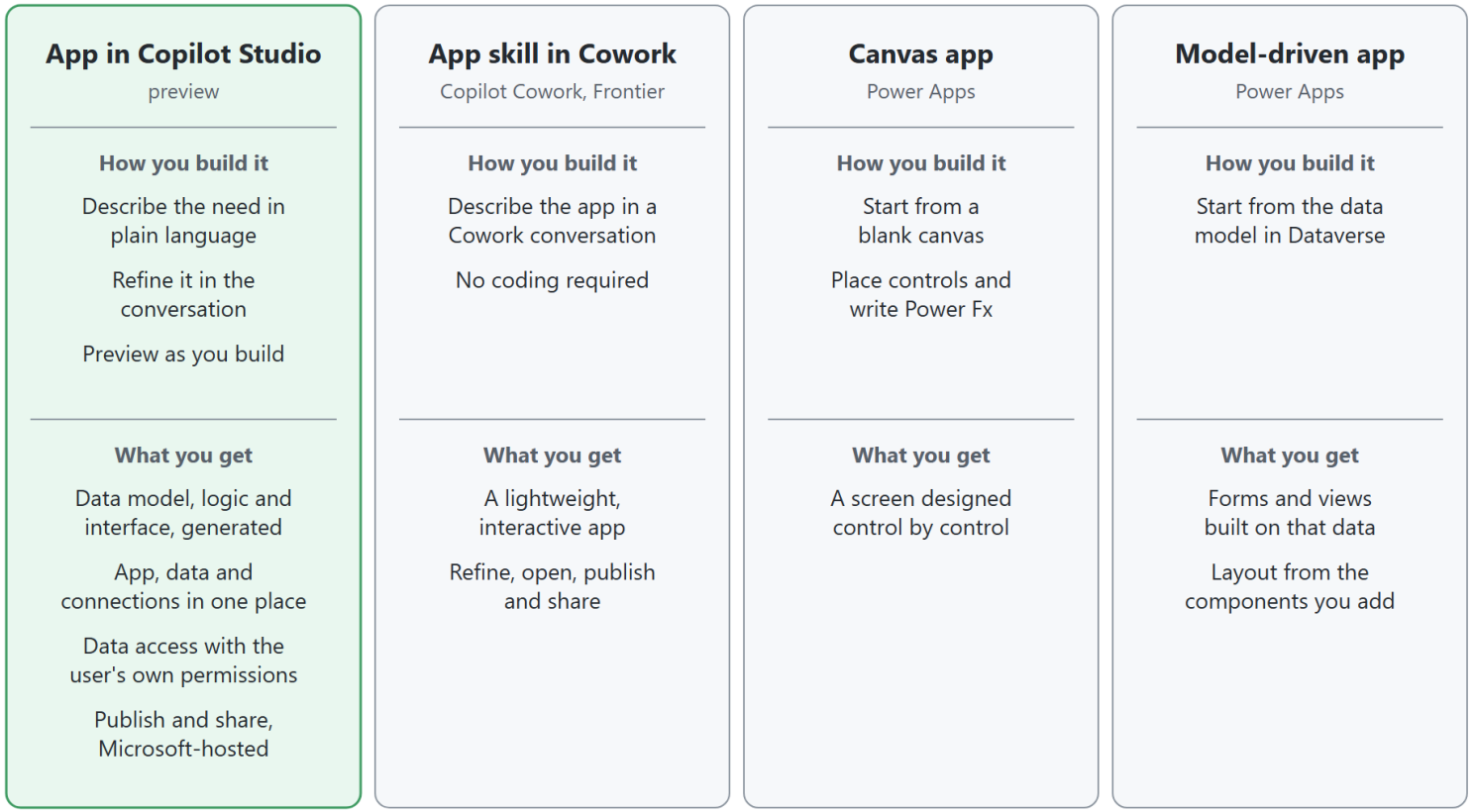
## A web app you describe, and Copilot Studio builds




An **app** is an interactive web experience: Copilot Studio generates its data model, logic and user interface from your description, and every change goes through the conversation

# How these apps compare

## The whole app from a description, next to Cowork and Power Apps

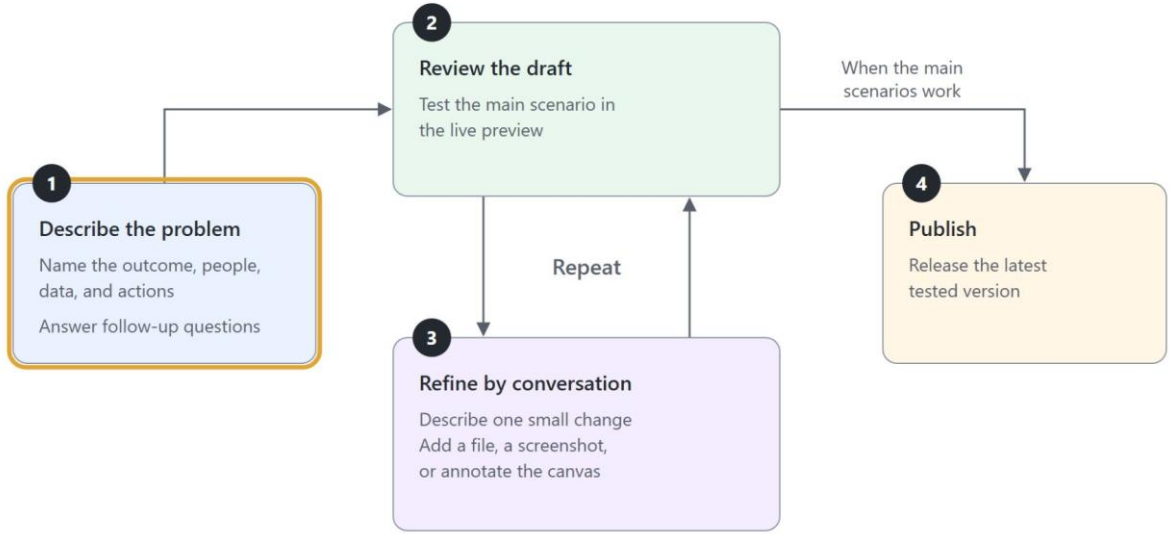



An app in Copilot Studio is generated whole from a description; a canvas app is designed control by control, a model-driven app starts from Dataverse tables, and Cowork's App skill builds a lighter app in a conversation

# Building apps

## Describe the app, then review and refine it until the main scenarios work

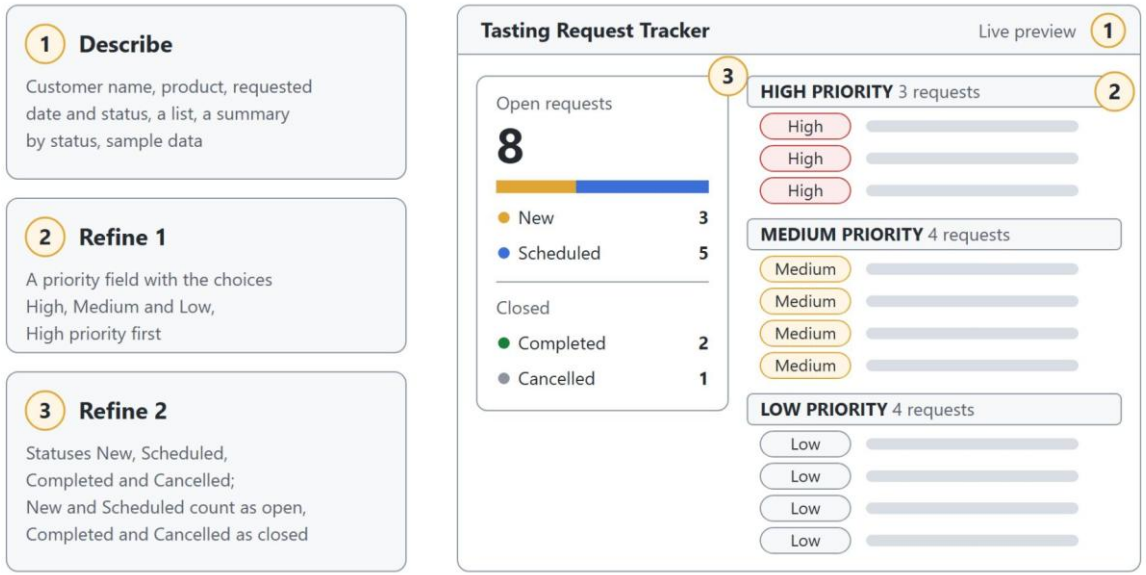
- **Describe**: name the outcome, people, data and actions; Copilot Studio generates the data model, logic and interface
- **Refine**: one small change per message, checked in the live preview; the code view is read-only
- **Data access**: granted on the SharePoint list or Dataverse table, separately from sharing the app
- **Copilot Credits**: billing starts as soon as you start building



# The Tasting Request Tracker

## An app refined by conversation, checked in the live preview

- **Describe**: one message named the people, the data and the actions; the first draft took about five minutes
- **Refine 1**: a priority field, with High requests listed first
- **Refine 2**: four statuses, with New and Scheduled counted as open
- **Check** each change in the preview before sending the next one



# Takeaways

## What to carry into publishing and maintaining

- Decide the **kind of agent** before you build: there is no conversion
- Read the **step list** and **History** to see how the orchestrator planned
- A skill's **description** and an MCP server's **description** decide when they are used
- Prove grounding in the **activity trace**, not in the wording of the reply
- Run the **same test set** before and after every change

